



Gemma 4

完全ガイド

AIで月100万円稼ぐ

完全攻略ガイド

セットアップから実装、営業まで

すべてをこの1冊に凝縮

生成日: 2026年4月10日



目次

1. セットアップガイド

- macOS / Windows / Linux / Raspberry Pi 対応
- ステップバイステップ手順

2. 実装コード集

- 建設業向け音声日報AI
- 医療・歯科向け診察メモAI
- 議事録AI、翻訳メディア、チャットボット、受付ロボット

3. トラブルシューティング

- 30+ エラー別解決方法
- FAQ、最後の手段

4. クイックリファレンス

- コマンド一覧、早見表、営業ヒッチテンプレート

5. PDFテンプレート集

- 提案資料、チェックシート、契約書、ROI計算シート
- 営業レポート、メールテンプレート



Gemma 4 完全セットアップガイド

【必須確認】ハードウェア要件

環境	CPU	RAM	GPU	推奨度
デスクトップ PC	i5以上	16GB以上	RTX 3060以上（あ ると速い）	★★★★
MacBook	M1/M2/M3	16GB以上	Apple Silicon内蔵	★★★★
ラップトップ	i7以上	16GB以上	内蔵GPU可	★★★
Raspberry Pi 5	ARM Cortex-A72	8GB以上	CPU推論	★ (軽量モデルのみ)

最小要件：RAM 8GB、ストレージ 20GB以上

【OS別】ステップバイステップガイド



macOS (Intel / Apple Silicon両対応)

Step 1: Homebrewの確認

```
brew --version
```

なければインストール：

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2: 必要なツールのインストール

```
# Python3
brew install python@3.11

# Node.js (オプション)
brew install node

# Git
brew install git
```

Step 3: Ollamaのインストール

```
# Homebrewで簡単インストール
brew install ollama

# または公式サイトから
# https://ollama.ai/download
```

Step 4: Ollamaサービスの開始

```
# バックグラウンドで起動
brew services start ollama

# 確認
ollama list
```

Step 5: Gemma 4のダウンロード

```
# 初回は5〜10分かかります
ollama pull gemma4

# ダウンロード完了確認
ollama list

# gemma4 latest 9b 9.2GB と表示されればOK
```

Step 6: 動作確認

```
ollama run gemma4 "こんにちは。あなたは何ですか?"
```

Step 7: Python環境の準備

```
# 仮想環境作成（推奨）
python3.11 -m venv gemma-env
source gemma-env/bin/activate

# ライブラリインストール
pip install --upgrade pip
pip install langchain ollama python-dotenv gradio openai-whisper torch transformers
```

Step 8: 動作確認 (Python)

```
python3 -c "
from langchain.llms import Ollama
llm = Ollama(model='gemma4')
print(llm('Hello, what is 2+2?'))
"
```

✓ macOS セットアップ完了

Windows 10/11

Step 1: WSL 2 のセットアップ（推奨）

```
# PowerShellを管理者で実行してから：
wsl --install
wsl --install -d Ubuntu-22.04
wsl --set-default-version 2
```

再起動が必要です

Step 2: WSL内でのセットアップ

```
# WSLを開く
wsl

# Ubuntu内で実行
sudo apt update
sudo apt install python3 python3-pip python3-venv curl git

# Pythonバージョン確認
python3 --version
```

Step 3: Ollamaのインストール

```
# WSL内で
curl -fsSL https://ollama.ai/install.sh | sh

# または Windows 用インストーラを使用
# https://ollama.ai/download → Windows版をダウンロード
```

Step 4: Gemma 4ダウンロード

```
ollama pull gemma4
```

Step 5: Python環境準備

```
# WSL内で
python3 -m venv ~/gemma-env
source ~/gemma-env/bin/activate
pip install --upgrade pip
pip install langchain ollama python-dotenv gradio openai-whisper torch transformers
```

Step 6: 動作確認

```
python3 -c "from langchain.llms import Ollama; llm = Ollama(model='gemma4'); print(llm('test'))"
```

✅ **Windows** セットアップ完了

Linux (Ubuntu / Debian)

Step 1: 必要なパッケージのインストール

```
sudo apt update
sudo apt install python3 python3-pip python3-venv curl git

# Python3.11がなければ
sudo apt install python3.11 python3.11-venv
```

Step 2: Ollamaのインストール

```
curl -fsSL https://ollama.ai/install.sh | sh

# 確認
ollama --version
```

Step 3: Ollamaサービスの開始

```
# systemdで自動起動設定
sudo systemctl start ollama
sudo systemctl enable ollama

# ステータス確認
sudo systemctl status ollama
```

Step 4: Gemma 4ダウンロード

```
ollama pull gemma4
```

Step 5: Python環境準備

```
python3.11 -m venv gemma-env
source gemma-env/bin/activate

pip install --upgrade pip
pip install langchain ollama python-dotenv gradio openai-whisper torch transformers
```

Step 6: 動作確認

```
python3 -c "from langchain.llms import Ollama; print(Ollama(model='gemma4')('Hi'))"
```

✅ **Linux** セットアップ完了

Raspberry Pi 5 (ARM64)

注意 ⚠️

- Raspberry Pi 4以下はメモリ不足のため非推奨
- Raspberry Pi 5 + 8GB RAM 以上が必須

- 全投稿生成は難しいが、推論（動作確認）は可能

Step 1: OSセットアップ

```
# Raspberry Pi OS (64-bit) を新規インストール
# https://www.raspberrypi.com/software/ からイメージをダウンロード

# インストール後、SSH接続
ssh pi@raspberrypi.local
```

Step 2: システム更新

```
sudo apt update
sudo apt upgrade -y

# 必要なライブラリ
sudo apt install -y python3 python3-pip python3-venv build-essential
```

Step 3: Ollamaのインストール

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Step 4: 軽量モデルのダウンロード

```
# Gemma 4は重いため、軽量版を推奨
ollama pull orca-mini:3b
# または
ollama pull neural-chat:7b
```

Step 5: Python環境

```
python3 -m venv gemma-env
source gemma-env/bin/activate

pip install langchain ollama python-dotenv gradio
# 注: torch/whisper は Pi 上でインストール困難なため、推論のみ
```

Step 6: バックグラウンド実行

```
# systemdサービスとして設定
sudo systemctl start ollama
sudo systemctl enable ollama
```

【共通】セットアップ後の確認

確認チェックリスト

```
# 1. Ollamaが動作しているか
ollama list

# 2. Gemma 4が使用可能か
ollama run gemma4 "テスト"

# 3. ローカルサーバーが立ち上がっているか
curl http://localhost:11434/api/tags

# 4. Pythonからアクセスできるか
python3 << 'EOF'
from langchain.llms import Ollama
llm = Ollama(model='gemma4')
print(llm('2 + 2 = ?'))
EOF
```

トラブル時の確認コマンド

```
# Ollamaプロセスの確認
ps aux | grep ollama

# ポート確認 (11434)
lsof -i :11434

# Gemmaモデルの再ダウンロード
ollama rm gemma4
ollama pull gemma4
```

【次ステップ】 **Web**インタフェース（オプション）

Open WebUI のインストール（全OS対応）

```
# Dockerが必要
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data --name open-webui ghcr.io/open-webui/open-webui:latest

# ブラウザで http://localhost:3000 を開く
```

簡易**WebUI**の構築（Python）

```
pip install gradio

# app.py を作成
cat > app.py << 'EOF'
import gradio as gr
from langchain.llms import Ollama

llm = Ollama(model='gemma4')

def chat(message):
    return llm(message)

gr.Interface(fn=chat, inputs='text', outputs='text').launch(share=True)
EOF

# 実行
python3 app.py
```



業界別実装コード集

【実装①】建設業向け：音声日報自動生成AI

必要なライブラリ

```
pip install openai-whisper gradio langchain ollama
```

コード例 : **constructiondailyreport.py**

```

import whisper
from langchain.llms import Ollama
import json
from datetime import datetime

# 音声ファイルをテキスト化
def transcribe_audio(audio_file_path):
    """MP3/WAVをテキストに変換"""
    model = whisper.load_model("base")
    result = whisper.transcribe(audio_file_path)
    return result["text"]

# テキストを日報に変換
def generate_daily_report(transcribed_text):
    """テキストから構造化日報を生成"""
    llm = Ollama(model='gemma4')

    prompt = f"""
    以下は建設現場の音声メモです。これから以下の形式で日報を作成してください。

    音声メモ：
    {transcribed_text}

    出力形式（JSON）：
    {{
        "date": "YYYY-MM-DD",
        "site_name": "現場名",
        "weather": "天候",
        "workers": "作業員数",
        "work_summary": "本日の作業内容（100字以内）",
        "completed_tasks": ["タスク1", "タスク2"],
        "tomorrow_plans": ["予定1", "予定2"],
        "safety_issues": "安全上の問題（なければなし）",
        "materials_needed": ["材料1", "材料2"]
    }}
    """

    response = llm(prompt)
    return response

# 実行例
if __name__ == "__main__":
    audio_file = "meeting.mp3"

    # Step 1: 音声をテキスト化
    text = transcribe_audio(audio_file)
    print(f"✅ 音声認識完了\n{text[:100]}...")

    # Step 2: 日報を生成
    report = generate_daily_report(text)
    print(f"✅ 日報生成完了\n{report}")

```

使用方法

```
python3 construction_daily_report.py
```

【実装②】医療・歯科向け：診察メモ自動生成 + 患者 FAQ

コード例：`medical_assistant.py`

```

from langchain.llms import Ollama
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
import json

class MedicalAssistant:
    def __init__(self):
        self.llm = Ollama(model='gemma4')
        # 患者情報は完全ローカル保管（外に出さない）
        self.patient_data = {}
        self.memory = ConversationBufferMemory()

    def generate_diagnosis_note(self, patient_info: dict, symptoms: str):
        """診察メモを自動生成（情報は一切外に出さない）"""

        prompt = f"""
患者情報（社内のみ）：
名前：{patient_info.get('name', '匿名')}
年齢：{patient_info.get('age', '不明')}
既往歴：{patient_info.get('history', 'なし')}

症状：{symptoms}

以下のフォーマットで診察メモを作成してください：
[診断メモ]
- 主訴：
- 診察所見：
- 診断：
- 処方/治療内容：
- 次回受診予定：
"""

        return self.llm(prompt)

    def generate_patient_friendly_explanation(self, diagnosis: str):
        """医学用語を患者向けに翻訳"""

        prompt = f"""
以下の医学診断を患者にわかりやすく説明してください。
難しい医学用語は避けて、日常言葉で説明してください。

医学診断：
{diagnosis}

患者向け説明：
"""

        return self.llm(prompt)

    def generate_faq_from_diagnosis(self, diagnosis: str):
        """診断から患者が聞きそうな質問と回答を自動生成"""

```

```
        prompt = f"""
以下の診断に基づいて、患者がよく聞く5つのFAQを作成してください。

診断：{diagnosis}

形式：
Q1: 質問内容？
A1: 回答

Q2: 質問内容？
A2: 回答

...
"""

        return self.llm(prompt)

# 使用例
if __name__ == "__main__":
    assistant = MedicalAssistant()

    patient = {
        'name': '田中太郎',
        'age': 45,
        'history': '高血圧'
    }

    # 診察メモを生成
    memo = assistant.generate_diagnosis_note(patient, "頭痛と首の凝りが3日続いている")
    print("📝 診察メモ：")
    print(memo)

    # 患者向け説明を生成
    explanation = assistant.generate_patient_friendly_explanation(memo)
    print("\n📖 患者向け説明：")
    print(explanation)

    # FAQを自動生成
    faq = assistant.generate_faq_from_diagnosis(memo)
    print("\n❓ よくあるご質問：")
    print(faq)
```

【実装③】完全ローカル議事録AI

コード例 : **meetingminutesai.py**

```

import whisper
from langchain.llms import Ollama
import json
from datetime import datetime

class MeetingMinutesAI:
    def __init__(self):
        self.whisper_model = whisper.load_model("base")
        self.llm = Ollama(model='gemma4')

    def transcribe_meeting(self, audio_file):
        """会議音声を文字起こし"""
        print(f"🎤 文字起こし中...")
        result = whisper.transcribe(audio_file)
        return result["text"]

    def extract_action_items(self, transcript):
        """TODOアクション項目を自動抽出"""

        prompt = f"""
以下の会議議事録から、実行可能なアクション項目を抽出してください。
各アクション項目に優先度と担当者を付けてください。

議事録：
{transcript}

出力形式：
■アクション項目
- [優先度:高/中/低] 内容（担当者） / 期限：○月○日
- [優先度:中] 内容（担当者） / 期限：○月○日
...
"""

        return self.llm(prompt)

    def summarize_meeting(self, transcript):
        """会議を3文で要約"""

        prompt = f"""
以下の会議議事録を3文以内で要約してください。

議事録：
{transcript}

要約：
"""

        return self.llm(prompt)

    def generate_minutes(self, audio_file, meeting_title):
        """完全な議事録を生成"""

```

```
# Step 1: 音声から文字起こし
transcript = self.transcribe_meeting(audio_file)

# Step 2: 要約
summary = self.summarize_meeting(transcript)

# Step 3: アクション項目抽出
actions = self.extract_action_items(transcript)

# ローカルファイルに保存（外部に出さない）
minutes = {
    "title": meeting_title,
    "date": datetime.now().isoformat(),
    "summary": summary,
    "actions": actions,
    "full_transcript": transcript
}

return minutes

# 使用例
if __name__ == "__main__":
    ai = MeetingMinutesAI()

    minutes = ai.generate_minutes(
        audio_file="meeting_recording.mp3",
        meeting_title="営業戦略会議 2024年4月"
    )

    # 結果をJSONで保存（社内PC内に）
    with open("minutes.json", "w", encoding="utf-8") as f:
        json.dump(minutes, f, ensure_ascii=False, indent=2)

    print("✅ 議事録を生成・保存しました")
    print(f"📌 要約: {minutes['summary']}")
    print(f"✅ アクション項目: {len(minutes['actions'])}個")
```

【実装④】 **AI**翻訳メディアの自動投稿エンジン

コード例 : **autotranslationmedia.py**

```

import feedparser
from langchain.llms import Ollama
import json
import hashlib
from datetime import datetime

class TranslationMediaEngine:
    def __init__(self):
        self.llm = Ollama(model='gemma4')
        self.processed_urls = set() # 既に翻訳した記事を追跡

    def fetch_foreign_articles(self, rss_feed_urls: list):
        """複数のRSSフィードから記事を取得"""
        articles = []

        for url in rss_feed_urls:
            print(f"📡 {url} から記事を取得中...")
            feed = feedparser.parse(url)

            for entry in feed.entries[:5]: # 最新5件
                article = {
                    'title': entry.title,
                    'url': entry.link,
                    'summary': entry.summary if hasattr(entry, 'summary') else '',
                    'published': entry.published if hasattr(entry, 'published') else ''
                }
                articles.append(article)

        return articles

    def translate_to_japanese(self, text):
        """英語から日本語に翻訳"""

        prompt = f"""
以下の英文を自然な日本語に翻訳してください。
技術用語は日本語カタカナで統一してください。

英文：
{text}

日本語翻訳：
"""

        return self.llm(prompt)

    def generate_japanese_headline(self, english_title):
        """英語タイトルから魅力的な日本語見出しを生成"""

        prompt = f"""
以下の英語タイトルから、日本のSNS（Threads/X）でバズりそうな
日本語見出しを3パターン生成してください。

```

```
英語タイトル: {english_title}
```

```
提案 (3パターン) :
```

```
1.  
2.  
3.  
"""
```

```
return self.llm(prompt)
```

```
def generate_article_body(self, title, summary):  
    """要約から1,000字程度の日本語記事本文を生成"""
```

```
    prompt = f"""
```

```
以下の情報から、日本人読者向けに1,000字程度の  
解説記事を作成してください。
```

```
タイトル: {title}
```

```
概要: {summary}
```

```
記事本文:
```

```
"""
```

```
return self.llm(prompt)
```

```
def create_social_media_snippet(self, article_title, article_body):  
    """Threads/X用の抜粋を生成"""
```

```
    prompt = f"""
```

```
以下の記事から、Threads/Xで拡散しそうな150字程度の  
抜粋を作成してください。絵文字も活用してください。
```

```
タイトル: {article_title}
```

```
記事: {article_body[:500]}
```

```
抜粋:
```

```
"""
```

```
return self.llm(prompt)
```

```
def process_articles(self, articles):  
    """記事をバッチ処理して日本語記事に変換"""
```

```
    japanese_articles = []
```

```
    for article in articles:
```

```
        article_id = hashlib.md5(article['url'].encode()).hexdigest()
```

```
        # 既に処理済みならスキップ
```

```
        if article_id in self.processed_urls:  
            continue
```

```

        print(f"🔄 変換中: {article['title']}")

        # Step 1: タイトル翻訳
        jp_title = self.generate_japanese_headline(article['title'])

        # Step 2: 本文生成
        jp_body = self.generate_article_body(jp_title, article['summary'])

        # Step 3: SNS用抜粋
        snippet = self.create_social_media_snippet(jp_title, jp_body)

        japanese_articles.append({
            'original_url': article['url'],
            'japanese_title': jp_title,
            'japanese_body': jp_body,
            'social_snippet': snippet,
            'published': datetime.now().isoformat()
        })

        self.processed_urls.add(article_id)

    return japanese_articles

def save_to_note(self, articles):
    """Note APIに投稿 (オプション) """

    for article in articles:
        filename = f"articles/{article['japanese_title'][:20]}.md"
        with open(filename, 'w', encoding='utf-8') as f:
            f.write(f"# {article['japanese_title']}\n\n")
            f.write(article['japanese_body'])
            f.write(f"\n\n[オリジナル]({article['original_url']})")

    print(f"✅ {len(articles)}件の記事を保存しました")

# 使用例
if __name__ == "__main__":
    engine = TranslationMediaEngine()

    # 翻訳対象のRSSフィード
    rss_feeds = [
        "https://news.ycombinator.com/rss", # Y Combinator
        "https://www.techcrunch.com/feed/", # TechCrunch
    ]

    # 実行
    articles = engine.fetch_foreign_articles(rss_feeds)
    jp_articles = engine.process_articles(articles)
    engine.save_to_note(jp_articles)

```

【実装⑤】AIテンプレート販売：ChatGPT風チャットボット（ローカル版）

コード例：`localchatbotui.py`

```

import gradio as gr
from langchain.llms import Ollama
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
import json

class LocalChatbotTemplate:
    def __init__(self, system_prompt=""):
        self.llm = Ollama(model='gemma4')
        self.memory = ConversationBufferMemory()
        self.system_prompt = system_prompt or "You are a helpful AI assistant."
        self.conversation_count = 0

    def chat(self, user_message):
        """会話ロジック"""

        full_prompt = f"""
{self.system_prompt}

ユーザー: {user_message}

アシスタント:
"""

        response = self.llm(full_prompt)
        self.conversation_count += 1

        return response

# Gradio UI
def create_gradio_interface():
    chatbot = LocalChatbotTemplate(
        system_prompt="You are a helpful AI assistant. Answer in Japanese when asked in Ja
    )

    def respond(message, chat_history):
        response = chatbot.chat(message)
        chat_history.append((message, response))
        return "", chat_history

    with gr.Blocks() as demo:
        gr.Markdown("# 🤖 ローカルChatGPT（API無料版）")
        gr.Markdown("このアシスタントはあなたのPC内で動作します。データは一切外に出ません。")

        chatbot_display = gr.Chatbot(label="会話")
        msg = gr.Textbox(
            label="メッセージ",
            placeholder="何か聞いてみてください...",
            lines=1
        )
        submit = gr.Button("送信")

```

```
        submit.click(respond, [msg, chatbot_display], [msg, chatbot_display])
        msg.submit(respond, [msg, chatbot_display], [msg, chatbot_display])

    return demo

# 実行
if __name__ == "__main__":
    interface = create_gradio_interface()
    interface.launch(share=True)
```

【実装⑥】 Raspberry Pi AI端末：受付ロボット化

コード例：`reception_robot.py`

```

import os
from langchain.llms import Ollama
import pyttsx3 # テキスト音声変換
import speech_recognition as sr # 音声認識

class ReceptionRobot:
    def __init__(self):
        self.llm = Ollama(model='gemma4')
        self.recognizer = sr.Recognizer()
        self.engine = pyttsx3.init()
        self.engine.setProperty('rate', 120) # 話す速度

    def listen_to_visitor(self):
        """訪問者の声を認識"""

        with sr.Microphone() as source:
            print("🎤 お話してください...")
            audio = self.recognizer.listen(source, timeout=10)

        try:
            text = self.recognizer.recognize_google(audio, language="ja-JP")
            return text
        except sr.UnknownAudioFrameError:
            return "申し訳ありませんが、よく聞き取れませんでした"

    def process_visitor_request(self, visitor_input):
        """訪問者のリクエストを処理"""

        prompt = f"""
あなたは会社の受付ロボットです。
訪問者の要件に応じて、簡潔で親切な応答をしてください。
社内のどこの部署に案内するか判断してください。

訪問者の発言: {visitor_input}

応答 (30字以内):
"""

        response = self.llm(prompt)
        return response

    def speak_response(self, response_text):
        """応答を音声で返す"""

        print(f"🗣️ 受付AI: {response_text}")
        self.engine.say(response_text)
        self.engine.runAndWait()

    def greeting(self):
        """初期グリーティング"""
        greeting = "いらっしゃいませ。本日のご用件をお話してください。"

```

```

        self.speak_response(greeting)

    def run(self):
        """受付ロボット起動"""

        self.greeting()

        while True:
            visitor_input = self.listen_to_visitor()

            if "終わり" in visitor_input or "ありがとう" in visitor_input:
                closing = "本日はご来訪ありがとうございました"
                self.speak_response(closing)
                break

            response = self.process_visitor_request(visitor_input)
            self.speak_response(response)

# 実行
if __name__ == "__main__":
    robot = ReceptionRobot()
    robot.run()

```

テンプレート販売向け複数モデル設定

```

templates = {
    "営業支援AI": {
        "system_prompt": "営業戦略と顧客対応についてアドバイスする営業支援AI",
        "price": 4980,
        "features": ["リード分析", "営業資料自動生成", "顧客対応シミュレーション"]
    },
    "マーケティングAI": {
        "system_prompt": "デジタルマーケティング戦略のアドバイスをする",
        "price": 4980,
        "features": ["SNS戦略立案", "コンテンツ案", "分析レポート自動化"]
    },
    "カスタマーサポートAI": {
        "system_prompt": "顧客サポート対応を支援するAI。常に丁寧で親切に",
        "price": 9980,
        "features": ["FAQ自動生成", "問い合わせ仕分け", "サポート品質管理"]
    },
}

```



トラブルシューティング完全版

エラー別解決ガイド

1 インストール・初期化エラー

✗ `ollama: command not found`

原因: Ollamaがインストールされていないか、PATHが通っていない

解決方法:

```
# macOS
brew install ollama

# Linux
curl -fsSL https://ollama.ai/install.sh | sh

# Windows
# https://ollama.ai/download から exe をインストール

# インストール後
exec $SHELL # シェルを再起動
ollama --version # 確認
```

✗ `Error: model not found` / `gemma4 not found`

原因: Gemma 4をダウンロードしていない

解決方法:

```
# ダウンロード開始（初回は10分程度）
ollama pull gemma4

# ダウンロード進捗を確認
ollama list

# 強制的に再ダウンロード
ollama rm gemma4
ollama pull gemma4
```

❌ Failed to connect to the server

原因: Ollamaサーバーが起動していない

解決方法:

```
# macOS (Homebrewで起動)
brew services start ollama

# Linux (systemdで起動)
sudo systemctl start ollama
sudo systemctl enable ollama # 自動起動設定

# Windows (GUIアプリを起動)
# Ollama.exeをダブルクリック

# 確認
curl http://localhost:11434/api/tags
```

2 メモリ・パフォーマンスエラー

❌ CUDA out of memory / Out of memory

原因: PCのメモリが足りない

解決方法:

```
# 1. 軽量モデルに切り替える
ollama pull orca-mini:3b
ollama run orca-mini:3b

# 2. バックグラウンドアプリを閉じる
# Task Manager (Windows) やActivity Monitor (Mac) で
# 不要なアプリを終了

# 3. メモリを確認
# macOS
vm_stat

# Linux
free -h

# Windows
Get-ComputerInfo -Property OsTotalVisibleMemorySize
```

✖ `Response time too slow` / 非常に遅い

原因: GPUが使われていない、またはスペック不足

解決方法:

```
# 1. GPU対応を確認
ollama list # VRAM情報を確認

# 2. CPU最適化を有効化
export OLLAMA_NUM_PARALLEL=1 # 並列処理を1に制限

# 3. より軽量なモデルを使用
ollama pull neural-chat:7b
```

3 Python・ライブラリエラー

✖ `ModuleNotFoundError: No module named 'langchain'`

原因: langchainがインストールされていない

解決方法:

```
# 仮想環境を使っているか確認
which python3 # 仮想環境内なら ~/..../gemma-env/... と表示

# インストール
pip install langchain

# 複数バージョンが競合している場合
pip uninstall langchain -y
pip install langchain==0.0.350
```

✖ `ConnectionRefusedError: [Errno 61] Connection refused`

原因: Ollamaサーバーに接続できない

解決方法:

```
# 1. サーバーが起動しているか確認
ps aux | grep ollama

# 2. ポート11434が使われているか確認
lsof -i :11434

# 3. サーバーを再起動
killall ollama
ollama serve # フォアグラウンドで起動
```

❌ torch not installed / CUDA関連エラー

原因: PyTorchがGPU対応でインストールされていない

解決方法:

```
# GPU対応版をインストール
pip uninstall torch -y

# NVIDIA GPU (CUDA)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu11

# AMD GPU (ROCm)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/rocm

# CPU only
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu

# Apple Silicon (M1/M2/M3)
pip install torch torchvision torchaudio
```

4 音声認識エラー (Whisper)

❌ Error running Whisper model

原因: Whisperモデルがダウンロードされていない、またはメモリ不足

解決方法:

```
# 軽量モデル「tiny」から始める
python3 << 'EOF'
import whisper
model = whisper.load_model("tiny")
result = whisper.transcribe("audio.mp3")
print(result["text"])
EOF

# より高精度が必要な場合
# "tiny" → "base" → "small" → "medium" → "large"
# サイズが大きいほど精度が上がるが、メモリも増加
```

❌ No audio input detected

原因: マイクが接続されていない、または認識されていない

解決方法:

```
# macOS: システム設定 → サウンド → 入力デバイスを確認
# Windows: 設定 → サウンド → 入力デバイスを確認

# Linux: 音声テスト
arecord -l # マイクを列挙

# Python テスト
python3 << 'EOF'
import speech_recognition as sr
recognizer = sr.Recognizer()
with sr.Microphone() as source:
    print("Listening...")
    audio = recognizer.listen(source)
    print(f"Audio captured: {len(audio.frame_data)} bytes")
EOF
```

5 ビジネス実装時のエラー

❌ JSON parse error / JSONパースエラー

原因: Gemmaの出力が予期したJSON形式ではない

解決方法:

```
# エラーハンドリングを追加
import json
from langchain.llms import Ollama

llm = Ollama(model='gemma4')

def safe_json_parse(response_text):
    try:
        # 最初のJSONブロックを抽出
        start = response_text.find('{')
        end = response_text.rfind('}') + 1

        if start >= 0 and end > start:
            json_str = response_text[start:end]
            return json.loads(json_str)
    except json.JSONDecodeError:
        # フォールバック：構造化されたテキストで返す
        return {"error": "JSON parse failed", "raw_response": response_text}

response = llm("返してください: {'name': 'test'}")
data = safe_json_parse(response)
```

Rate limiting / API制限エラー

原因: 短時間に大量の推論リクエストを送信

解決方法:

```
import time
from langchain.llms import Ollama

llm = Ollama(model='gemma4')

# レート制限を実装
def batch_process_with_throttle(items, delay=1):
    results = []
    for item in items:
        response = llm(item)
        results.append(response)
        time.sleep(delay) # 1秒待機
    return results

# 使用
items = ["質問1", "質問2", "質問3"]
results = batch_process_with_throttle(items, delay=0.5)
```

✖ File permission denied / ファイル権限エラー

原因: 出力ファイルが保存できない

解決方法:

```
# macOS/Linux
chmod 755 output_directory/
chmod 644 output_file.txt

# ディレクトリを作成して権限を設定
mkdir -p output/
chmod 777 output/

# Python で事前に確認
import os
output_dir = "output/"
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
    os.chmod(output_dir, 0o755)
```

6 Raspberry Pi 固有エラー

✖ Illegal instruction / 非合法命令エラー

原因: Pi 上で最適化されていないバイナリが実行されている

解決方法:

```
# ARMv6 互換版をインストール
pip install --upgrade pip
pip install ollama --no-binary :all:

# または軽量モデルのみ使用
ollama pull neural-chat:7b
```

✖ SWAP memory full / スワップメモリ不足

原因: メモリが完全に満杯になった

解決方法:

```
# スワップサイズを増加 (Pi 設定)
sudo nano /etc/dphys-swapfile
# CONF_SWAPSIZE=2048 に変更 (デフォルト100MB)

# 反映
sudo /etc/init.d/dphys-swapfile restart

# または軽量モデルのみ実行
ollama pull orca-mini:3b
```

よくある質問 (FAQ)

Q1: 複数のユーザーが同時にアクセスできる？

A: Ollamaサーバーはマルチユーザー対応。ただし推論は1件ずつ処理されるため、複数リクエストは待機キューに入ります。

```
# 非同期処理で高速化
import asyncio
from langchain.llms import Ollama

async def process_parallel(queries):
    tasks = [llm(q) for q in queries]
    return await asyncio.gather(*tasks)
```

Q2: 生成されたテキストがおかしい場合？

A: プロンプト設計を改善してください。

```
# ❌ 悪い例
llm("日報を作成して")

# ✅ 良い例
llm("""
以下の形式で日報を作成してください：
- 日付：
- 天気：
- 作業内容：
- 明日の予定：

本文：
""")
```

Q3: セキュリティはどうなっている？

A: Ollamaはローカル実行のため、データはPC内に留まります。ネットワークを介さない限り外部へ流出しません。

```
# 設定例：ローカルホストのみアクセス
# ~/.ollama/ollama のポートを127.0.0.1に制限
export OLLAMA_HOST=127.0.0.1:11434
```



最後の手段：リセット

どうしてもうまくいかない場合：

```
# 1. Ollamaを完全に削除
brew uninstall ollama # macOS
# または Windows/Linux は同様にアンインストール

# 2. キャッシュをクリア
rm -rf ~/.ollama/

# 3. 再インストール
brew install ollama # macOS など

# 4. Gemma 4を新規ダウンロード
ollama pull gemma4
```

Gemma 4 クイックリファレンス

コマンド一覧 (コピペ用)

セットアップ (1回だけ)

```
# インストール
brew install ollama # macOS
curl -fsSL https://ollama.ai/install.sh | sh # Linux

# モデルダウンロード
ollama pull gemma4

# 動作確認
ollama run gemma4 "テスト"
```

Python 最小コード

```
from langchain.llms import Ollama
llm = Ollama(model='gemma4')
print(llm("質問"))
```

日報 AI (建設業)

```
python3 construction_daily_report.py
```

議事録 AI

```
python3 meeting_minutes_ai.py
```

翻訳メディア

```
python3 auto_translation_media.py
```

チャットボット UI

```
python3 local_chatbot_ui.py
# → http://localhost:7860 を開く
```

受付ロボット (Raspberry Pi)

```
python3 reception_robot.py
```

トラブル対応早見表

症状	コマンド
Ollamaが起動しない	<code>brew services start ollama</code>
モデルが見つからない	<code>ollama pull gemma4</code>
Python が遅い	<code>ollama rm gemma4; ollama pull orca-mini:3b</code>
メモリ不足	<code>export OLLAMA_NUM_PARALLEL=1</code>
完全リセット	<code>ollama rm gemma4; ollama cache clear</code>

ビジネス別ファイル構成

```
Gemma4-Business/
├─ construction_daily_report.py    # 建設業
├─ medical_assistant.py            # 医療
├─ meeting_minutes_ai.py           # 議事録
├─ auto_translation_media.py        # 翻訳メディア
├─ local_chatbot_ui.py              # チャットボット
├─ reception_robot.py               # 受付ロボット
├─ output/
│   └─ reports/                    # 生成ファイル保存
│   └─ logs/                       # ログ
└─ config.json                     # 設定ファイル
```

営業用ピッチテンプレート（30秒版）

建設業向け

「現場の日報、毎日手書きですか？
実は、音声で話しかけるだけで日報が自動生成される
システムがあるんです。

月9,800円で導入でき、現場監督の事務作業が50%削減できます」

医療向け

「患者情報をクラウドに送るのはセキュリティリスク。
でも、社内PCだけで動くAIなら安全です。

診察メモ自動生成で記録時間が40%削減、月額29,800円です」

企業全体向け

「毎回の会議で議事録作成に時間がかかってませんか？
音声から自動でアクション項目も抽出。

完全ローカルなので情報漏洩の心配なし。月29,800円です」

環境構築チェックリスト

- ☐ Ollama インストール済み
- ☐ Gemma 4 ダウンロード済み
- ☐ Python 3.11+ インストール済み
- ☐ 仮想環境構築済み
- ☐ ライブラリインストール完了
- ☐ テスト実行成功

全てチェック済みなら準備**OK**！

月額課金ビジネス料金設定ガイド

ビジネスモデル	推奨価格	最小企業数	月額目標
業界特化AIツール	¥9,800	10社	¥98,000
議事録AI	¥5,000	20社	¥100,000
コンサル（AIエージェント）	¥50,000	4社	¥200,000
テンプレート販売	¥4,980	20本	¥99,600

初期営業活動（Week 1）

- ターゲット企業リスト作成（20社）
- LinkedIn メッセージ送信（1日2〜3社）
- メール営業開始（1日1社）
- 紹介経由営業（既知の人脈5人に連絡）

目標：トライアル申し込み3件 / 初受注1件

ROI計算テンプレート

年間削減額：¥ _____, 000

年間コスト：¥ _____, 600

純利益： ¥ _____, 400

ROI： _____%

よくある質問（FAQ）

Q: プログラミング未経験でもできる？ **A:** 提供するコードはそのまま使えます。ただしカスタマイズには Python の基礎知識が必要です。

Q: セキュリティは大丈夫？ **A:** ローカル実行のため、データは PC 内に留まります。最高のセキュリティです。

Q: 導入後のサポートは？ **A:** 3ヶ月の無料サポート + 月額サポートプラン (¥9,800/年)

Q: 複数企業への展開は？ **A:** 各企業で独立した環境として動作します。パートナー展開も可能。

次のステップ

1. **Week 1:** セットアップ + 市場調査
2. **Week 2:** プロトタイプ完成 + デモ作成
3. **Week 3:** 営業資料 + 営業開始
4. **Week 4:** 初受注取得

目標：30日以内に初受注 ¥50,000



PDF用テンプレート集

テンプレート①：提案資料（**A4**サイズ）

🚀 完全ローカル AI ツール導入提案

～ 情報漏洩ゼロ・月額課金で即導入 ～

📊 現状の課題

- ✗ 毎日の日報作成に1時間以上かかっている
- ✗ 議事録を手書きしている
- ✗ クラウドに情報を送るのが心配
- ✗ 音声入力で記録したい

💡 ソリューション：Gemma 4 ローカル AI

- ✓ 社内PC内で完全動作
- ✓ データが一切外に出ない
- ✓ インターネット不要
- ✓ 月9,800円～

🎯 導入メリット

事務作業 50% 削減	
セキュリティ リスク ゼロ	
即日導入可能	
サポート無料（3ヶ月）	

💰 料金表

月額プラン：¥9,800
年間プラン：¥98,000（1ヶ月分割引）
エンタープライズ：カスタム見積

- ※ セットアップ無料
- ※ サポート3ヶ月無料
- ※ その後 ¥9,800/年の保守費用

トライアル申込

1. メール or LINE で「デモ希望」とお送りください
2. 30分のオンライン デモを実施
3. 無料トライアル（14日間）を開始

連絡先：example@example.com

LINE：@example-id

テンプレート②：実装チェックシート

Gemma 4 ビジネス化 実装チェックシート

プロジェクト名 : _____

開始日 : _____年 ____月 ____日

担当者 : _____

Phase 1: セットアップ

- ☐ Ollamaインストール完了
- ☐ Gemma 4ダウンロード完了
- ☐ Python環境構築完了
- ☐ テスト実行成功

進捗 : _____ 100%

Phase 2: 開発

- ☐ プロトタイプコード完成
- ☐ 3人にデモ実施
- ☐ フィードバック反映
- ☐ UI完成

進捗 : _____ 50%

Phase 3: 営業準備

- ☐ ピッチ資料作成
- ☐ 提案メールテンプレート
- ☐ デモビデオ撮影
- ☐ 営業リスト作成

進捗 : _____ 0%

Phase 4: 営業活動

- ☐ 営業メール送信 (20社)
- ☐ LinkedIn営業 (10人)
- ☐ 紹介経由営業 (5人)
- ☐ 初受注獲得

進捗 : _____ 0%

全体進捗： 38%

 メモ

目標月額：¥500,000

必要受注数：10社 × ¥50,000

当月実績：¥0

来月目標：¥50,000（初受注1社）

3ヶ月目目標：¥300,000（6社）

テンプレート③：顧客契約書（簡易版）

AI ツール導入サービス 契約書

甲（提供者）：_____

乙（利用者）：_____

契約日：____年 __月 __日

第1条 サービス内容

甲は以下のサービスを乙に提供する：

サービス名：_____

機能：

- _____
- _____
- _____

第2条 料金

月額料金：¥ _____, 000

支払い方法：銀行振込（毎月末日）

支払い期日：翌月5日

※ 初期セットアップ無料

※ 3ヶ月間のサポート無料

第3条 秘密保持

甲乙双方は、本契約に関連する情報を
第三者に開示しないものとする。

第4条 免責

本サービスはローカル環境で動作するため、
データ漏洩の責任は負わない。

ただし、セットアップ時の初期設定エラーは
無料に対応する。

第5条 契約期間

開始日：____年 __月 __日
終了日：____年 __月 __日（1年）

更新：自動更新（30日前に申し出で解約可）

契約署名

甲（提供者）

氏名：_____
署名：_____
日付：____年 __月 __日

乙（利用者）

企業名：_____
代表者：_____
署名：_____
日付：____年 __月 __日

テンプレート④：ROI計算シート

AI ツール導入効果 ROI 計算シート

【導入前の状況】

現在の業務：日報作成

1日あたりの作業時間： 60分
1ヶ月の作業日数： 20日
1ヶ月の合計時間： 1,200分 (20時間)

人件費計算：

時給：¥2,000/時間
1ヶ月の人件費：¥40,000

【導入後の効果】

作業時間削減率：50%

削減時間：600分/月 (10時間)
削減費用：¥20,000/月

【コスト】

AI ツール月額料金：¥9,800

年額：¥117,600

【ROI 計算】

年間削減額：¥20,000 × 12 = ¥240,000

年間コスト：¥117,600

純利益：¥240,000 - ¥117,600 = ¥122,400

ROI：122,400 ÷ 117,600 = 104%

【結論】

- ✓ 初月から赤字にならない
- ✓ 2ヶ月目以降は利益
- ✓ 年間¥122,400の削減効果

推奨：即導入

テンプレート⑤：月次レポート（営業成績表）

Gemma 4 ビジネス 月次営業レポート

2024年 4月

当月の営業成績

営業メール送信数：	20社
返信率：	15% (3社)
デモ実施数：	2社
トライアル申込：	1社
初受注：	0社
当月売上：	¥0

営業活動内容

Week 1

- ターゲット企業リスト作成 (20社)
- 営業メール送信開始

Week 2

- LinkedIn メッセージ送信 (10人)
- 返信メール返答 (3件)

Week 3

- デモ実施 (1社：医療クリニック)
- デモ実施 (1社：建設業)

Week 4

- フォローアップメール送信 (3社)
- トライアル契約 (医療クリニック)

所感・課題

【良かった点】

-  LinkedIn営業の返信率が高い (40%)
-  デモを見せると説得力が上がる
-  ROI計算シートが決定打になった

【改善点】

- ✕ メール件名をもっと工夫する
- ✕ 業界別のピッチを作成すべき
- ✕ 営業資料をビジュアル化する

来月の目標

- ☐ 営業メール送信：30社
- ☐ デモ実施：3社
- ☐ トライアル申込：2社
- ☐ 初受注：1社（¥50,000）
- ☐ 月間売上目標：¥50,000

Next Action

1. 業界別営業資料を3種類作成
 2. デモビデオを YouTube にアップ
 3. パートナー営業を5人に声かけ
 4. Threads で毎日ノウハウ発信開始
-

テンプレート⑥：メール営業テンプレート

件名：【15分で日報が完成】建設現場AI導入のご提案

本文：

〇〇様

お疲れ様です。

〇〇（あなた）と申します。

現場の日報作成に時間がかかっているのを
もしご存知でしたら、
うちの AI ツールが役に立つかもしれません。

【現在の課題】

現場監督が毎日 1 時間かけて日報を手書き
→ そのデータを事務所で PC に打ち込み直し
→ 二重作業になっている

【うちのツール】

監督が現場で「今日の作業は...」と
音声で話しかけるだけで、
5 分で日報が完成します。

- ・天候、作業員数、完了タスク、安全課題まで自動抽出
 - ・情報は現場 PC 内に完全ローカル保存
 - ・セキュリティリスク ゼロ
-

【導入メリット】

- ✓ 日報作業が 80% 削減
- ✓ 月額 ¥9,800（初月セットアップ無料）
- ✓ 年間で ¥122,000 の削減効果
- ✓ 3 ヶ月無料サポート付き

【30 分無料デモのご案内】

実際に動かしてみることをお勧めします。
15 分見れば「これ使える」って感覚が分かります。

→ デモ申込：こちらのリンク or メール返信で

今、3 社限定で無料トライアル（14 日間）
やってます。

ご興味あれば、返信いただけたら幸いです。

よろしくお願いします。

〇〇（あなたの名前）

Gemma 4 完全ガイド | AIで月100万円稼ぐ完全攻略

このドキュメントに含まれる全ての情報は教育目的です。
ビジネス導入前に十分な検証と専門家への相談をお勧めします。